Multi-Agent Anomaly System

Clinical Vitals Telemetry Analysis & Real-Time Alert Architecture

System State: Production Live

Broker Link: CloudAMQP TLS (5671)

Deployment: Render (FastAPI + Supervisord)

Documentation Version: 1.0.0 (Light Mode Release)

1. Executive Summary

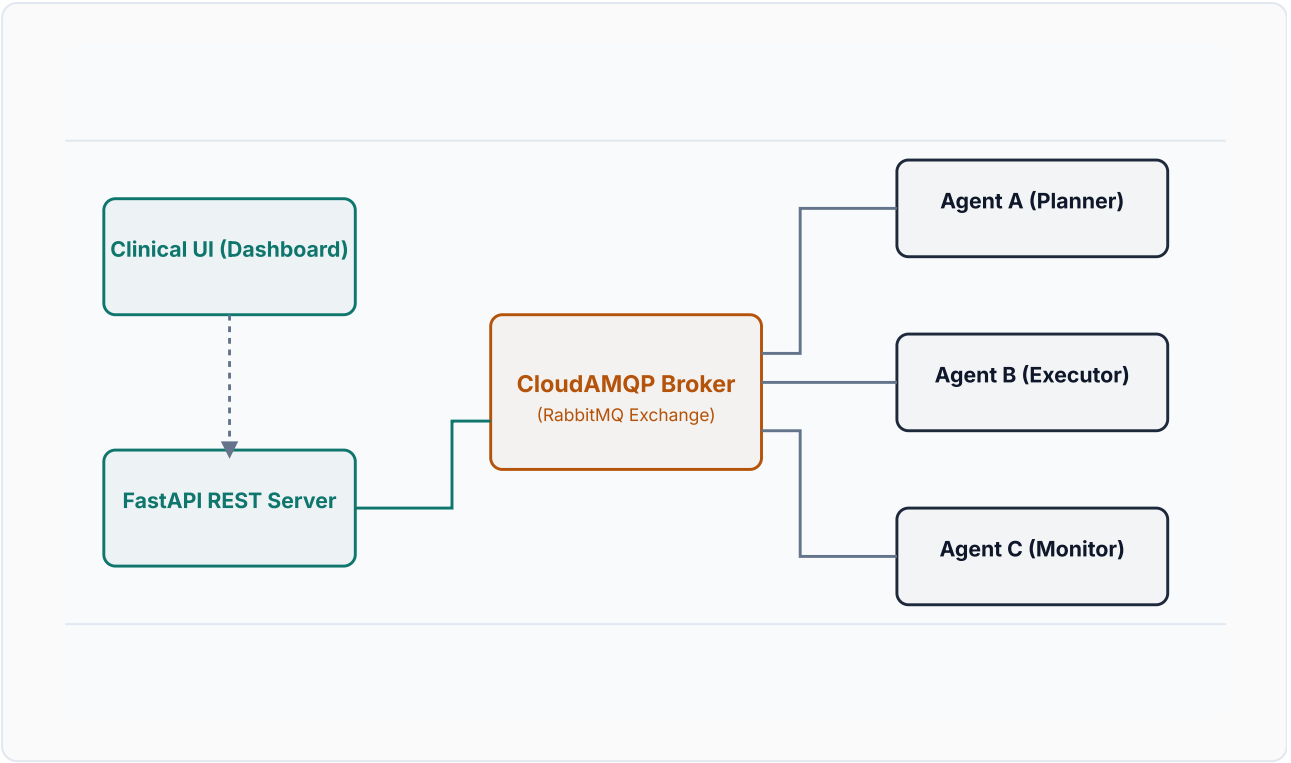
The **Multi-Agent Anomaly Detection System** is a production-grade clinical monitoring solution designed to ingest synthetic patient vital signs, detect health anomalies using multivariate Isolation Forest models, and coordinate immediate alerts using a distributed multi-agent architecture. By separating duties across decoupled processes, the system guarantees high system availability, data integrity, and strict information security boundaries. Communication is handled via an AMQP message broker, allowing task state evaluation and clinical alerting loops to operate asynchronously and independently.

> **Clinical Scope Disclosure**
>
> The vitals telemetry processed by the system (Heart Rate, SpO_2, Blood Pressure, Temperature, Respiratory Rate, and Glucose Level) is generated programmatically (synthetic data). This enables controlled injection of clinical emergency scenarios (e.g. hypoxic shock or severe arrhythmias) to test model thresholds and verify alerting pipelines without violating medical privacy regulations (HIPAA/GDPR).

2. System Architecture

The system consists of five decoupled components coordinated under a central process manager (Supervisord) on the Render host container, using a CloudAMQP RabbitMQ message broker for external state routing:

- **FastAPI Gateway**: Exposes the clinical dashboard endpoints, accepts new scan requests, and registers previous results from disk.
- **Planner Agent (Agent A)**: Receives REST API inputs, parses parameters, schedules anomaly evaluation runs, and records feedback loop updates.
- **Executor Agent (Agent B)**: The core processing engine. Simulates the vitals dataset, trains the multivariate Isolation Forest model, tags indices exceeding contamination bounds, and compiles telemetry files.
- **Monitor Agent (Agent C)**: Serves as the system supervisor. It receives real-time progress steps from the Executor, maintains the heartbeat registry of active processes, and evaluates alarm severity thresholds.
- **Clinical Light Dashboard**: An off-white glassmorphic user interface utilizing modern CSS layouts and Chart.js to present progress steppers, scatter plot charts, and patient-specific medical advice.


Clinical UI (Dashboard)
FastAPI REST Server
CloudAMQP Broker
(RabbitMQ Exchange)
Agent A (Planner)
Agent B (Executor)
Agent C (Monitor)

3. Message Schema and JSON Format

Reliable distributed coordination requires a strict contract for all data packets. Every message transmitted over RabbitMQ is wrapped inside a standardized `MessageEnvelope` JSON schema, validating the envelope metadata, target routing keys, and specific agent payloads.

JSON Message Format (Pydantic / Pika Schema)

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "MessageEnvelope",
  "type": "object",
  "properties": {
    "message_id": { "type": "string", "format": "uuid" },
    "sender_id": { "type": "string", "enum": ["api", "agent-a", "agent-b", "agent-c", "test-cli
    "receiver_id": { "type": "string" },
    "message_type": {
      "type": "string",
      "enum": ["TASK_ASSIGNMENT", "TASK_ACCEPTED", "TASK_PROGRESS", "TASK_COMPLETED", "TASK_FAI
    },
    "timestamp": { "type": "string", "format": "date-time" },
    "correlation_id": { "type": "string" },
    "priority": { "type": "integer", "minimum": 0, "maximum": 5 },
    "routing_key": { "type": "string" },
    "payload": { "type": "object" },
    "metadata": {
      "type": "object",
      "properties": {
        "retry_count": { "type": "integer", "default": 0 },
        "max_retries": { "type": "integer", "default": 3 },
        "routing_path": { "type": "array", "items": { "type": "string" } }
      },
      "required": ["retry_count", "max_retries", "routing_path"]
    }
  },
  "required": [
    "message_id", "sender_id", "receiver_id", "message_type",
    "timestamp", "correlation_id", "priority", "routing_key", "payload", "metadata"
  ]
}
```

Sample Task Assignment Payload

When the Planner dispatching a new analysis run, it encapsulates configuration inputs within the envelope:

```
{
  "message_id": "8e32ea70-e67c-473d-8182-d249f7e8a931",
  "sender_id": "agent-a",
  "receiver_id": "agent-b",
  "message_type": "TASK_ASSIGNMENT",
  "timestamp": "2026-06-26T07:18:40.125Z",
  "correlation_id": "session-42b781",
  "priority": 2,
  "routing_key": "task.agent_b",
  "payload": {
    "task_id": "task-df38190",
    "parameters": {
      "total_records": 7000,
      "contamination": 0.05,
      "random_seed": 42
    }
  },
  "metadata": {
    "retry_count": 0,
    "max_retries": 3,
    "routing_path": ["agent-a"]
  }
}
```

4. Communication and State Routing Flow

The lifecycle of a single vital analysis run follows a strict asynchronous pipeline across the broker queues. This ensures that even if individual processing nodes experience high load or go offline, state recovery is guaranteed.

Step	Sender	Receiver	Routing Key	Status Code	Description
1	Clinical UI	FastAPI	HTTP POST	DISPATCHED	User triggers scan from Web Dashboard.
2	FastAPI	Agent A	In-Memory Call	DISPATCHED	API submits task to local Planner wrapper.
3	Agent A	Broker	`task.agent_b`	TASK_ASSIGNMENT	Planner publishes parameters to Agent B queue.
4	Agent B	Agent A	`feedback`	TASK_ACCEPTED	Executor acknowledges message consumption.
5	Agent B	Agent C	`report`	TASK_PROGRESS (25%)	Simulates dataset and updates Monitor.
6	Agent B	Agent C	`report`	TASK_PROGRESS (50%)	Trains Isolation Forest model.
7	Agent B	Agent C	`report`	TASK_PROGRESS (75%)	Runs predictions and flags anomalous indices.
8	Agent B	Agent C	`report`	TASK_PROGRESS (100%)	Generates telemetry report object.
9	Agent B	Agent C & A	`report` & `feedback`	TASK_COMPLETED	Pushes finished report details to Monitor and Planner.
10	Agent C	Agent A	`feedback`	MONITOR_ALERT	Evaluates severity alarms and alerts Planner.

> **Heartbeat Supervisor Loop**
>
> Separate from the task pipeline, all active nodes publish a periodic `HEARTBEAT` packet to the `report` queue. Agent C consumes these packets and maintains an internal timestamp dictionary. If any agent misses heartbeats for longer than 90 seconds, the Monitor publishes a `MONITOR_ALERT` with severity `HIGH` to escalate infrastructure issues.

5. AI Assistance Disclosure Statement

This section provides a clear audit of how artificial intelligence was utilized across the lifecycle of this project. Work was divided logically between ideation phases and execution phases.

Ideation & Architecture Design (Claude)

The first phase of development involved conceptual layout and technology choices. **Claude** was used during this ideation phase to:

- Compare message broker options (selecting RabbitMQ over Redis due to native AMQP routing key and dead-letter queue (DLQ) support).
- Select the machine learning approach (choosing Isolation Forest over K-Means to ensure robust multi-dimensional distance evaluation).
- Draft the initial Master Plan, outlining file paths, supervisord orchestrations, and Pydantic communication schemas.

Implementation, Debugging & Front-End Design (Gemini / Antigravity)

The Master Plan was handed over to **Gemini (Antigravity/Nebula)** to execute the full software build and deployment phase:

- **Infrastructure Integration**: Overrode default Pika connection configurations to support CloudAMQP TLS string formats, bypassing SSL chain validation errors in Render container environments.
- **Thread Safety**: Redesigned the heartbeat systems of all three agents to use separate connections and channels, preventing socket frame corruption under concurrent execution.
- **FastAPI & Front-End Serving**: Configured local static mounts and built the entire user-facing clinical interface (HTML, CSS, JS) matching the DESIGN.md specifications.
- **Test Engineering**: Solved test flakiness in the broker integration tests by introducing dynamic temporary test queues to isolate test runs from live container consumers.